Exercise 3 - Hints and solutions

The **hints** come first. Try to solve each step with the help of the hints.

The **solutions** are at the bottom of this notebook. Look at them only after you have tried!

Hints

Part 1 - The samples

1. Read a sheet

```
samples = pd.read_excel(file, sheet_name='...')
samples
```

The sheet names are `samples` and `DE_results`.

2. New column

Divide the column by one million and give the result a new column name:

```
samples['new_column'] = samples['old_column'] / 1000000
```

3. Bar plot

```
samples.plot(kind='bar', x='...', y='...', title='...',
xlabel='...', ylabel='...')
```

The x axis is the sample name, the y axis is the number of million reads.

4. The strange sample

Look at your bar plot: which bar is much lower? Then look at `percent_mapped` for the same sample. A low mapping rate often means the RNA was degraded or the sample was contaminated.

5. Mean per condition

Use `groupby`, like in the lecture:

```
df.groupby('column to group by')['column to calculate'].mean()
```

Part 2 - The results

6. Size and missing values

- `.shape` gives `(rows, columns)`. One row is one gene.
- In the output of `.info()`, look at **Non-Null Count**. A column with fewer values than the number of rows has missing values.

7. VEGFA

- Select the row of `VEGFA`.
- Is `log2FoldChange` positive (up) or negative (down)?
- Calculate `2 ** log2FoldChange` with the number you see, e.g. `2 ** 3.5`.

8. Significant genes

```
significant = results[results['padj'] < 0.05]
```
Use `len()` or `.shape` to count the rows.

9. Up and down

Two conditions must be true at the same time, so use `&` and put each condition in brackets:

```
up = results[(condition 1) & (condition 2)]
```
For the last question, look at the `log2FoldChange` of the significant genes that are neither in `up` nor in `down`

10. Top 10

- Sort `up` with `.sort_values('log2FoldChange', ascending=False)`: biggest first.
- `.head(10)` gives the first 10 rows.
- Select columns with a list: `[['gene', 'log2FoldChange', 'padj']]`.

You can chain everything in one line.

11. Histogram

```
results.plot(kind='hist', y='...', bins=30, title='...',
xlabel='...')
```
Where is the highest bar? Around which value of `log2FoldChange`?

Solutions

In [ ]:
```
import pandas as pd

file = '../../downloads/hypoxia_rnaseq.xlsx' # Change the relative path if r
```

Part 1 - The samples

1.

In [ ]:
```
samples = pd.read_excel(file, sheet_name='samples')
samples
```

2.

In [ ]:
```
samples['million_reads'] = samples['total_reads'] / 1000000
samples
```

3.

In [ ]:
```
samples.plot(kind='bar', x='sample', y='million_reads',
             title='Reads per sample', xlabel='Sample', ylabel='Reads (milli
```

4. Sample `H2` has only about 13 million reads, half of the other samples, and only 71% of its reads map to the genome (the others: 94-95%). Something went wrong with this sample, for example degraded RNA, contamination or a problem during sequencing.

5.

In [ ]:
```
samples.groupby('condition')['million_reads'].mean()
```

The hypoxia mean is lower because of sample `H2`.

Part 2 - The results

6.

In [ ]:
```
results = pd.read_excel(file, sheet_name='DE_results')
results.shape
```

There are **198 genes**.

In [ ]:
```
results.info()
```

`log2FoldChange`, `pvalue` and `padj` have missing values. `padj` has the most: DESeq2 does not calculate it for genes with very low expression.

7.

In [ ]:
```
results[results['gene'] == 'VEGFA']
```

In [ ]:
```
2 ** 3.952
```

`VEGFA` is about **15 times higher** in hypoxia, as expected for a gene that helps make new blood vessels when oxygen is low.

8.

In [ ]:
```
significant = results[results['padj'] < 0.05]
len(significant)
```

9.

In [ ]:
```
up = results[(results['padj'] < 0.05) & (results['log2FoldChange'] > 1)]
down = results[(results['padj'] < 0.05) & (results['log2FoldChange'] < -1)]

print('Up:', len(up))
print('Down:', len(down))
```

35 + 18 = 53, but there are 75 significant genes. The other 22 genes are significant but change **less than 2 times**.

10.

In [ ]:
```
up.sort_values('log2FoldChange', ascending=False).head(10)[['gene', 'log2Fol
```

11.

In [ ]:
```
results.plot(kind='hist', y='log2FoldChange', bins=30,
             title='Fold changes of all genes', xlabel='log2 fold change')
```

Most genes are around `0`: they do not change. Only a few genes are far from 0, on both sides.